ROS 2 - Sistema Operativo de Robótica

Proyecto de investigación - Sistemas de Comunicaciones

Pablo Sebastian Gomez · Irina Terebiznik · Matias Peña

Sistemas de Comunicaciones



Universidad Tecnológica Nacional
Facultad Regional Avellaneda
Departamento de Ingeniería Electrónica

Índice

1. Capítulo 1 **3**

1.1. Introducción del trabajo . . . 3
1.2. Objetivo y alcance . . . 3
1.3. Metodología y Fuentes de investigación . . . 4
1.4. Delimitación del estudio . . . 5
1.5. Universo Espacio Temporal . . . 5

2. Capítulo 2 **5**

2.1. Introducción del tema . . . 5
2.2. Descripción del contexto del tema . . . 6
2.3. Mercado y Segmentación del tema . . . 7
2.4. Legislación del tema . . . 8

3. Capítulo 3 **9**

3.1. Fundamentos técnicos . . . 9
Conceptos arquitectónicos: nodos y *topics* . . . 9
Patrones de comunicación: *services* y *actions* . . . 9
3.2. Ámbitos de aplicación . . . 10
3.3. Estrategias Competitivas . . . 10
3.4. Normativas asociadas . . . 10

4. Capítulo 4 **10**

4.1. Conclusiones . . . 10
4.2. Glosario de términos . . . 10
4.3. Referencias . . . 11

Extracto

El presente informe analiza ROS (*Robot Operating System*) desde la perspectiva de los sistemas de comunicaciones. Se describen sus conceptos arquitectónicos y patrones de comunicación, y se examina la transición desde la arquitectura híbrida de ROS 1, dependiente de un proceso maestro central, hacia el modelo descentralizado de ROS 2, sustentado en el estándar DDS y en el protocolo RTPS. Se analizan, asimismo, los mecanismos de calidad de servicio (QoS), la capa de abstracción de *middleware* (RMW) y sus principales implementaciones, el soporte para sistemas de tiempo real y microcontroladores, y el esquema de distribuciones del proyecto. El estudio se complementa con una prueba de concepto sobre hardware dedicado.

1. Capítulo 1

1.1. Introducción del trabajo

El presente trabajo de investigación se desarrolla en el marco de la asignatura Sistemas de Comunicaciones de la Universidad Tecnológica Nacional. Su propósito es analizar ROS (*Robot Operating System*) desde la perspectiva de los sistemas de comunicaciones, considerando los mecanismos de intercambio de información que emplea y los estándares sobre los que se sustenta.

Los sistemas robóticos actuales constituyen, en esencia, sistemas distribuidos. Sensores, actuadores, unidades de procesamiento y estaciones de supervisión operan como procesos independientes, frecuentemente alojados en distintos equipos, que deben intercambiar datos de manera continua a través de una red. En este contexto, la arquitectura de comunicaciones deja de ser un aspecto secundario y pasa a condicionar directamente la latencia, la confiabilidad y la escalabilidad del sistema en su conjunto.

ROS constituye uno de los *frameworks* de código abierto más difundidos para el desarrollo de software robótico, tanto en el ámbito académico como en el industrial. Su evolución resulta particularmente relevante para el estudio de las comunicaciones: la primera versión (ROS 1) se basa en una arquitectura híbrida que depende de un proceso maestro central, mientras que ROS 2 adopta un esquema descentralizado sustentado en el estándar DDS (*Data Distribution Service*) y en el protocolo RTPS (*Real-Time Publish-Subscribe*), que opera habitualmente sobre UDP/IP. Este trabajo aborda dicha transición, analizando sus fundamentos técnicos, la configuración de la calidad de servicio (QoS) y las implementaciones de *middleware* disponibles.

1.2. Objetivo y alcance

El objetivo general del trabajo es analizar el estado del arte de ROS, con foco en la evolución de su arquitectura de comunicaciones desde ROS 1 hasta ROS 2 y en los estándares y protocolos que la sustentan.

Para ello se plantean los siguientes objetivos específicos:

- Describir los conceptos arquitectónicos de ROS (nodos y *topics*) y sus patrones de comunicación (*services* y *actions*).

- Analizar la arquitectura híbrida de ROS 1 y las limitaciones derivadas de su dependencia del proceso maestro (ROS Master) como punto único de falla.
- Describir el modelo descentralizado de ROS 2 y su pila de protocolos: DDS, RTPS y UDP/IP.
- Analizar las políticas de calidad de servicio (QoS) y su configuración.
- Relevar el soporte de ROS 2 para sistemas operativos de tiempo real (RTOS) y para microcontroladores mediante micro-ROS.
- Describir la capa de abstracción de *middleware* (RMW) y comparar sus principales implementaciones: Eclipse Cyclone DDS, eProsima Fast DDS y RTI Connext DDS.
- Presentar el esquema de distribuciones de ROS y su vinculación con las versiones de Ubuntu.
- Realizar una demostración práctica sobre hardware dedicado (Raspberry Pi) que ilustre el intercambio de mensajes entre nodos de ROS 2 a través de la red.

En cuanto al alcance, el estudio se circunscribe a las capas de *middleware* y de transporte de datos, y se complementa con una prueba de concepto de carácter demostrativo. Quedan fuera de su alcance los algoritmos de percepción, planificación y control, así como el desarrollo de aplicaciones robóticas específicas.

1.3. Metodología y Fuentes de investigación

La investigación es de tipo descriptivo y de enfoque cualitativo. Se basa en el análisis documental de las fuentes técnicas que definen la arquitectura de ROS y de los estándares de comunicaciones sobre los que se apoya. Los resultados del análisis se contrastan mediante una prueba de concepto sobre hardware dedicado.

Las fuentes consultadas se agrupan en las siguientes categorías:

- **Documentación oficial de ROS:** documentación de ROS~2 y propuestas de mejora (REP, *ROS Enhancement Proposals*), en particular la REP~2000, que define las plataformas soportadas por cada distribución.
- **Especificaciones de estándares:** especificaciones de la OMG (*Object Management Group*) para DDS y DDSI-RTPS, y documentos RFC de la IETF para los protocolos UDP (RFC~768) e IPv4 (RFC~791).
- **Documentación de fabricantes:** documentación técnica de las implementaciones Eclipse Cyclone DDS, eProsima Fast DDS y RTI Connext DDS, y del proyecto micro-ROS.
- **Publicaciones académicas:** artículos científicos sobre el diseño, la arquitectura y el desempeño de ROS 2 y DDS.

1.4. Delimitación del estudio

El análisis se delimita según los siguientes criterios:

- **Sectores:** telecomunicaciones, robótica y dispositivos IoT (*Internet of Things*).
- **Objeto de estudio:** el subsistema de comunicaciones de ROS 1 y ROS 2, con exclusión del resto de las herramientas del ecosistema.
- **Capas analizadas:** *middleware* (RMW y DDS), protocolo de interoperabilidad (RTPS) y transporte (UDP/IP).
- **Plataformas:** distribuciones de ROS sobre Ubuntu Linux, sistemas operativos de tiempo real y microcontroladores mediante micro-ROS.

1.5. Universo Espacio Temporal

En el plano temporal, el estudio abarca la evolución de ROS desde el inicio de su desarrollo en 2007 hasta las distribuciones vigentes de ROS 2 en 2026. Se pone especial énfasis en el período 2021–2026, durante el cual ROS 2 supera a ROS 1 en adopción y este último alcanza el fin de su soporte con la distribución Noetic, en mayo de 2025.

En el plano espacial, el análisis tiene alcance global, dado que ROS y los estándares asociados son desarrollados y adoptados a nivel internacional. Se incorporan referencias al contexto argentino cuando resultan pertinentes.

2. Capítulo 2

2.1. Introducción del tema

ROS (*Robot Operating System*) es un *framework* de código abierto para el desarrollo de software robótico. A pesar de su denominación, no constituye un sistema operativo en sentido estricto: se ejecuta sobre un sistema operativo anfitrión, principalmente distribuciones de Linux, y provee una capa de *middleware* junto con un conjunto de bibliotecas y herramientas. Su función central es permitir que los distintos componentes de un robot se desarrollen como procesos independientes y se comuniquen entre sí mediante interfaces estandarizadas.

El proyecto se originó en 2007 en el Laboratorio de Inteligencia Artificial de la Universidad de Stanford, en el marco del proyecto STAIR (*STanford Artificial Intelligence Robot*), y continuó su desarrollo en la empresa Willow Garage. Actualmente está a cargo de la Open Source Robotics Foundation (OSRF) y, desde 2024, su gobernanza técnica es ejercida por la Open Source Robotics Alliance (OSRA), con la participación de una amplia comunidad internacional.

Los principales factores que explican su adopción son:

- **Modularidad:** cada función del robot se implementa como un componente independiente, lo que facilita el desarrollo en paralelo y el mantenimiento.

- **Reutilización:** el ecosistema dispone de un amplio conjunto de paquetes para navegación, percepción, manipulación y controladores de hardware.
- **Abstracción de hardware:** los componentes se comunican a través de interfaces normalizadas, con independencia del fabricante del sensor o actuador.
- **Herramientas de desarrollo:** provee utilidades de visualización, registro de datos, simulación y depuración.
- **Licenciamiento abierto:** el núcleo de ROS 1 se distribuye bajo licencia BSD y el de ROS 2 bajo licencia Apache 2.0. Ambas son licencias permisivas que habilitan su uso académico y comercial.

El diseño de ROS 1 se orientó al caso de uso del robot PR2 de Willow Garage, cuyas características determinantes eran: un único robot, recursos de cómputo de clase estación de trabajo, ausencia de requisitos de tiempo real, conectividad de red de alta calidad y aplicaciones predominantemente de investigación. A medida que su uso se extendió a entornos industriales, sistemas multirrobot y redes inalámbricas de menor confiabilidad, estas hipótesis de diseño se tornaron restrictivas. En respuesta, se desarrolló ROS 2, rediseñado desde su base sobre estándares industriales de comunicaciones.

La diferencia fundamental entre ambas versiones reside en su arquitectura de comunicaciones. ROS 1 emplea un esquema híbrido: un proceso maestro central (ROS Master, iniciado mediante el comando `roscore`) actúa como servicio de registro y resolución de nombres a través de XML-RPC, mientras que el intercambio de datos se realiza directamente entre nodos mediante protocolos propios (TCPROS y UDPROS). En consecuencia, el proceso maestro constituye un punto único de falla: sin él, los nodos no pueden localizarse entre sí ni establecer nuevas comunicaciones. Asimismo, ROS 1 ofrece opciones limitadas para ajustar el comportamiento de la comunicación, sin un modelo de calidad de servicio comparable al de DDS.

ROS 2, en cambio, adopta una arquitectura descentralizada basada en el estándar DDS (*Data Distribution Service*) y su protocolo de interoperabilidad RTPS. Esta elección elimina la dependencia de un proceso central, dado que el descubrimiento de participantes se realiza de forma distribuida, y habilita la configuración de políticas de calidad de servicio (QoS). Su diseño contempla, además, la ejecución en sistemas con requisitos de tiempo real y la integración de microcontroladores mediante micro-ROS. Asimismo, incorpora mecanismos de seguridad basados en la especificación DDS-Security, sin equivalente nativo en ROS 1.

Con la finalización del soporte de ROS 1 en 2025, ROS 2 se ha consolidado como la plataforma vigente para el desarrollo de nuevos sistemas robóticos.

Los aspectos técnicos de cada arquitectura se desarrollan en detalle en el Capítulo 3.

2.2. Descripción del contexto del tema

ROS se emplea como capa de integración en sistemas robóticos de diversa complejidad, desde plataformas educativas y de investigación hasta vehículos autónomos y robots industriales. En es-

tos sistemas, los componentes de software suelen distribuirse entre varios equipos: computadoras embarcadas en el robot, microcontroladores asociados a sensores y actuadores, y estaciones de supervisión remotas. ROS actúa como el *middleware* que permite a estos componentes intercambiar información de forma transparente respecto de su ubicación física en la red.

El software de ROS se publica en **distribuciones**: conjuntos versionados de paquetes que se prueban y se mantienen en conjunto. La primera distribución de ROS 2 se publicó en diciembre de 2017 y, durante sus primeros años, las versiones se lanzaron con una frecuencia semestral. Desde 2020 se publica una nueva distribución por año, en el mes de mayo. Las distribuciones de los años pares son de soporte extendido (LTS, *Long Term Support*) y reciben actualizaciones durante cinco años, mientras que las de los años impares tienen un ciclo de soporte de aproximadamente un año y medio. En paralelo existe Rolling, una distribución de desarrollo continuo en la que se incorporan los cambios antes de su inclusión en una versión estable.

Cada distribución está vinculada a una versión LTS específica de Ubuntu, que constituye su plataforma principal de soporte (*Tier 1*), según lo establecido en la REP 2000. Este vínculo se debe a que los paquetes binarios de ROS se compilan y se prueban contra las bibliotecas del sistema de esa versión de Ubuntu. En consecuencia, la elección de la distribución de ROS determina la versión del sistema operativo a utilizar. Las distribuciones LTS más recientes y su correspondencia son:

- **ROS 1 Noetic Ninjemys** (2020): Ubuntu 20.04. Última distribución de ROS 1, con fin de soporte el 31 de mayo de 2025.
- **ROS 2 Humble Hawksbill** (2022): Ubuntu 22.04, con soporte hasta mayo de 2027.
- **ROS 2 Jazzy Jalisco** (2024): Ubuntu 24.04, con soporte hasta mayo de 2029.
- **ROS 2 Lyrical Luth** (2026): Ubuntu 26.04, con soporte hasta mayo de 2031.

Además de Ubuntu, las distribuciones de ROS 2 ofrecen distintos niveles de soporte para otras plataformas, como Windows y Red Hat Enterprise Linux.

2.3. Mercado y Segmentación del tema

Al tratarse de un proyecto de código abierto y libre distribución, ROS no constituye un producto comercial en sí mismo. Su mercado se define, en cambio, por el ecosistema de organizaciones que lo adoptan como base para sus desarrollos y por las empresas que ofrecen productos y servicios compatibles.

A nivel mundial, la adopción de ROS puede segmentarse según los siguientes criterios:

- **Por sector de aplicación:** robótica industrial y colaborativa, logística y almacenes (robots móviles autónomos), vehículos autónomos, agricultura de precisión, robótica de servicio y salud, y aplicaciones espaciales.
- **Por tipo de usuario:** universidades y centros de investigación, empresas emergentes que desarrollan productos robóticos, grandes empresas industriales y fabricantes de hardware que proveen controladores compatibles con ROS para sus sensores, actuadores y robots.

- **Por versión:** si bien ROS 1 mantuvo una amplia base instalada durante más de una década, la adopción se orientó progresivamente hacia ROS 2: hacia septiembre de 2024, ROS 2 ya representaba casi el 80 % de las descargas totales de ROS. La finalización del soporte de ROS 1 en 2025 consolidó esta tendencia.

La adopción industrial se apoya en iniciativas como el consorcio ROS-Industrial, orientado a extender ROS para la automatización de la manufactura, y Autoware, una plataforma de código abierto para la conducción autónoma construida sobre ROS 2. Asimismo, la constitución de la OSRA en 2024 incorporó un modelo de membresía que suma la participación y el financiamiento de empresas del sector, como NVIDIA, Intrinsic y Qualcomm.

En Argentina, el uso de ROS se concentra principalmente en el ámbito académico, en universidades y grupos de investigación en robótica, automatización y control. En el ámbito productivo, su aplicación presenta un desarrollo incipiente, con mayor potencial en sectores como la agroindustria, la automatización de procesos industriales y la logística.

2.4. Legislación del tema

No se ha identificado legislación que regule específicamente a ROS ni a los *middleware* de comunicaciones robóticas. Sin embargo, los sistemas construidos sobre ROS quedan alcanzados por la normativa general aplicable a máquinas, productos con componentes digitales y tratamiento de datos. Las normas técnicas asociadas se desarrollan en la Sección 3.4.

En el contexto internacional, la Unión Europea constituye la principal referencia regulatoria:

- **Reglamento (UE) 2023/1230 relativo a las máquinas:** reemplaza a la Directiva 2006/42/CE y será de aplicación a partir del 20 de enero de 2027. Incorpora requisitos sobre el software que cumple funciones de seguridad y sobre la protección frente a ciberataques que puedan comprometer la operación segura de la máquina.

- **Reglamento (UE) 2024/2847 de Ciberresiliencia:** establece requisitos de ciberseguridad para productos con elementos digitales. Entró en vigor en diciembre de 2024 y será plenamente aplicable a partir del 11 de diciembre de 2027. Su alcance comprende a los robots conectados en red. Prevé un régimen diferenciado para el software de código abierto desarrollado sin fines comerciales.

- **Reglamento (UE) 2024/1689 de Inteligencia Artificial:** resulta aplicable a los sistemas robóticos que incorporan componentes de inteligencia artificial, según su nivel de riesgo.

En Argentina no se ha identificado legislación sancionada que regule específicamente la robótica. Los sistemas robóticos quedan alcanzados por normas de carácter general:

- **Ley 19.587 de Higiene y Seguridad en el Trabajo** (1972) y su decreto reglamentario 351/79, aplicables a la incorporación de robots en entornos laborales.

- **Ley 25.326 de Protección de los Datos Personales** (2000), aplicable cuando los robots capturan y transmiten información que permite identificar a personas, como imágenes o

audio.

- **Ley 11.723 de Propiedad Intelectual**, que comprende al software tras la modificación introducida por la Ley 25.036 (1998), y que da el marco legal a las licencias de código abierto bajo las que se distribuye ROS.

3. Capítulo 3

3.1. Fundamentos técnicos

Conceptos arquitectónicos: nodos y *topics*

Un sistema basado en ROS se organiza como un grafo de procesos que se comunican entre sí, denominado grafo de ROS. La unidad básica de este grafo es el **nodo**: un participante que realiza una función lógica específica, como leer un sensor, estimar la posición del robot o controlar un motor. Cada nodo utiliza una biblioteca cliente para comunicarse con el resto del sistema: `roscpp` y `rospy` en ROS 1, y `rclcpp` y `rclpy` en ROS 2, para los lenguajes C++ y Python respectivamente. Los nodos pueden comunicarse con otros nodos alojados en el mismo proceso, en otro proceso o en otro equipo de la red. Las conexiones entre ellos se establecen mediante un mecanismo de descubrimiento, cuya implementación difiere entre ROS 1 y ROS 2, según se detalla más adelante.

El mecanismo de comunicación fundamental es el ***topic***: un canal con nombre sobre el que opera un modelo de publicación/suscripción. Los nodos productores de datos (*publishers*) publican mensajes en un *topic*, y todos los nodos suscritos a él (*subscribers*) los reciben. Un mismo *topic* admite cero o más publicadores y cero o más suscriptores, lo que configura un esquema de comunicación de muchos a muchos. Este modelo presenta dos propiedades relevantes:

- **Anonimato:** el suscriptor no necesita conocer qué nodo originó el dato. Esto permite reemplazar, agregar o quitar nodos sin modificar el resto del sistema.
- **Tipado fuerte:** cada *topic* transporta un único tipo de mensaje, definido en un archivo `.msg`. A partir de esta definición, las herramientas de ROS generan automáticamente el código para los distintos lenguajes soportados.

Los *topics* resultan adecuados para flujos de datos continuos, como las lecturas de sensores o el estado del robot, donde no se espera una respuesta por parte del receptor.

Patrones de comunicación: *services* y *actions*

Además de los *topics*, ROS ofrece dos patrones de comunicación orientados a la interacción entre un cliente y un servidor.

Un ***service*** implementa una llamada a procedimiento remoto (RPC, *Remote Procedure Call*) bajo un esquema de solicitud y respuesta. Un nodo cliente envía una solicitud, el nodo servidor realiza el procesamiento correspondiente y devuelve un resultado. Su interfaz se define en un archivo `.srv`, compuesto por un mensaje de solicitud y uno de respuesta. Cada nombre de *service* debe

tener un único servidor, aunque puede ser utilizado por cualquier cantidad de clientes. Dado que el cliente generalmente queda a la espera del resultado, los *services* están pensados para operaciones de ejecución breve, como consultar un estado o modificar una configuración.

Una ***action*** es una llamada a procedimiento remoto de larga duración, que incorpora la posibilidad de recibir retroalimentación durante la ejecución y de cancelar el objetivo antes de su finalización. Un ejemplo típico es la orden de desplazar el robot hasta una posición determinada: la tarea puede demorar segundos o minutos, durante los cuales el servidor informa el progreso y el cliente puede cancelarla. Su interfaz se define en un archivo `.action`, compuesto por tres mensajes: objetivo (*goal*), resultado (*result*) y retroalimentación (*feedback*).

En ROS 2, las *actions* no constituyen un mecanismo de transporte independiente, sino que se construyen sobre los dos patrones anteriores: cada *action* se compone internamente de tres *services* (envío del objetivo, cancelación y obtención del resultado) y dos *topics* (retroalimentación y estado). En ROS 1, en cambio, las *actions* no formaban parte del núcleo y se implementaban mediante una biblioteca independiente denominada `actionlib`.

3.2. Ámbitos de aplicación

Donde se utiliza y como, en que mercados, etc.

3.3. Estrategias Competitivas

¿Competencia? ¿Producto único?

3.4. Normativas asociadas

¿Qué normas tiene asociadas ITU, IEEE?

4. Capítulo 4

4.1. Conclusiones

Al finalizar el ...

4.2. Glosario de términos

Terminología utilizada en el informe:

- **ROS 2:** *Robot Operating System 2*. Segunda generación de ROS, *framework* de código abierto para el desarrollo de software robótico.
- **Nodo:** Participante del grafo de ROS que realiza una función lógica específica y se comunica con otros nodos.

4.3. Referencias

[1] A. S. Sedra y K. C. Smith, *Microelectronic Circuits*, 7ma ed. Oxford University Press, 2015.

[2] Texas Instruments, *LM317 3-Terminal Adjustable Regulator*, Datasheet SLVS044X, 2020. [Disponible en carpeta `references/`].